Amauri Hopewell

Southern New Hampshire University

CS-499 Computer Science Capstone

October 4, 2026

Milestone Three: Algorithms and Data Structures Enhancement

1. What is the artifact, and when was it created?

This is the ABCU course catalog from CS-300 DSA: Analysis and Design, 2025 C-3 (May–June 2025). The original is Proj2.cpp in the original/ folder of this zip and at https://github.com/AmauriHopewell/CS300. It is one C++ file. It loads a CSV of course number, title, and a list of prerequisite numbers, keeps a row only if every prerequisite exists in the file, and used to store everything in a std::vector. Search was a linear scan. Print-all walked through that vector after stable_sort and printed out the contents. In Project 1 I also wrote runtime analysis for a hash table and a binary search tree, but I did not put those structures in the shipped code. This milestone implements that, plus the defects the Module Two review found in the same file.

2. Why this artifact, and how was it improved?

I picked this file because the original assignment was my data-structure choice. The vector program works for a few hundred advising rows, but it does not show that I can change the structure when needed to search through bigger databases. The Module One plan was to keep the loader and the menu, keep the vector as the CS-300 baseline, add a BST and a hash map keyed by course number, print all with an in-order walk, search with the hash map, and time all three. That is what is in enhanced/.

CourseBST has an insert, find, in-order walk, and a destructor that frees nodes. Search uses std::unordered_map, which is the STL hash table. Compare mode (menu 4) runs the same lookup

50,000 times on the vector, the tree, and the hash, and prints the time in microseconds. Menu 5 prints the Big-O table in the program so the demo does not depend on me reciting it. Menu 6 prints a topological schedule with Kahn's algorithm so a course never appears before its prerequisites. If there is a tie, they are resolved in order of course number.

I also fixed the bugs that made timings misleading. The original CSV loop never pushed the last cell unless the line ended with a comma; CSCI250 in ABCU_Advising_Program_Input.csv has no trailing comma and still loads with prerequisite CSCI101. operator== and operator> now match their names. The prereq sort uses Course& so the sort is kept. file.is_open is checked after open. The menu reads a local int, not argc. A row whose prerequisite is missing (BADPREREQ) is dropped. If I had inserted the already-sorted vector into the BST, the tree would have been a linked list, but this tree is filled median-first from that sorted vector so its height stays logarithmic (Cormen et al., 2009, pp. 286, 300).

For tests on my laptop using 129 courses and an optimized build, hash was the fastest structure. For ELEC120, which sits late in the vector, 50,000 trials took about 24,500 µs on the vector, 3,400 µs on the BST, and 450 µs on the hash.

3. Course outcomes from Module One

This improvement shows outcomes 3, 2 and 4. Outcome 3 is in the running program: vector versus BST versus hash, with times and a table, plus topological order as the leftover item from Figure 2. Outcome 2 is accomplished through the compare-mode output, menu 5, and this write-up. Outcome 4 is accombplished by the C++, chrono, and unordered_map used as tools instead of guessing. Outcome 1 will be satisfied by a GitHub branch after I take comments. Also, there is some progress towards Outcome 5 here: is_open, std::string, no C buffers. However, the security work is more substantial in the CS-320 auth filter and the CS-340 credential correction.

4. What I learned, and what got in the way

The first problem was the parser. The original character loop reset the cell on a comma and never flushed the leftover string, so the last field vanished, which slightly impacted search timing. The sample row CSCI250 exists so that problem can be detected. The prereq sort was for (Course c : courses), which copies. I switched to Course& to preserve sorting.

The BST fill would have caused an error in comparison mode. Inserting keys in sorted order creates a configuration that might not match a real-world program, so "O(log n)" on the screen would have been a linear pointer walk, and slower than the vector because the vector is contiguous. To fix that, I used median-first insert from the sorted array.

Also, even though I marked using topological ordering as optional in the original plan, I had some time and added it after the other three structures were working. It uses Kahn's algorithm, which is O(V + E). If the emitted order is shorter than the catalog, the program reports a cycle and stops. This makes sure a printed schedule never lists a course before its own prerequisites.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press. https://www.cs.mcgill.ca/~akroit/math/compsci/Cormen%20Introduction%20to%20Algorithms.pdf

Acknowledgment of generative AI use

I used Grok 4.6 (xAI) to help organize this narrative against the Milestone Three prompts and to format the Word file, and to find the page in the Cormen pdf to support the logarithmic height proposition.